

TeamNote of dbgusdn012

Compiled on March 28, 2025

Contents	7	Tips/Tools	15
1 Graph	2		
1.1 Flow	2		
1.2 Biparate Matching	3		
1.3 MCMF	3		
1.4 Bellman-Ford	4		
1.5 SCC (Strongly Connected Component)	4		
1.6 2-SAT	5		
2 Tree	6		
2.1 HLD	6	7.1 PBDS	15
2.2 Centroid	7		
3 Data Structure	8		
3.1 Merge Sort Tree	8		
3.2 Lazy Segment Tree (Sum ver.)	8		
3.3 Mo’s Algorithm	9		
4 Strings	10		
4.1 KMP	10		
4.2 Z	10		
4.3 Manacher	10		
4.4 Suffix Array & LCP	11	7.2 Bitmasking	16
5 Geometry	12		
5.1 CCW	12		
5.2 Line Intersection	12		
5.3 Convex Hull	12		
5.4 Rotating Callipus	13		
6 Extra	14		
6.1 Sparse Table	14		
6.2 Prefix Sum(Right Triangle Shape)	14		
6.3 Matrix Pow	14	7.3 Memo	16

1 Graph

1.1 Flow

//백준 1420<학교 가지마!> 소스코드

```
#define INF 0x7f7f7f7f
```

```
struct Edge
{
    int nxt, c = 0, f = 0;
    Edge* reverse;
```

```
void flow(int f1)
{
    f += f1;
    reverse->f -= f1;
}
```

```
};
```

```
int st, en;
string s[101];
vector<Edge*> stage[25000];
Edge* path[25000];
int parent[25000];
```

```
int dirx[] = {1, -1, 0, 0};
int diry[] = {0, 0, 1, -1};
```

```
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
```

```
int n, m;
cin >> n >> m;
```

```
for(int i = 0; i < n; i++)
{
    cin >> s[i];
    for(int j = 0; j < m; j++)
    {
        if(s[i][j] == 'K')
            st = (i*m+j)*2;
        else if(s[i][j] == 'H')
            en = (i*m+j)*2 + 1;
    }
}
```

```
for(int i = 0; i < n; i++)
    for(int j = 0; j < m; j++)
    {
        int cur_in = (i*m+j)*2, cur_out = (i*m+j)*2 + 1;
```

```
Edge* e1 = new Edge(), *e2 = new Edge();
e1->nxt = cur_out; e1->reverse = e2;
e2->nxt = cur_in; e2->reverse = e1;
```

```
if(s[i][j] == '.')
    e1->c = 1;
```

```
else if(s[i][j] != '#')
    e1->c = INF;
stage[cur_in].push_back(e1);
stage[cur_out].push_back(e2);
```

```
for(int k = 0; k < 4; k++)
{
    int di = i + dirx[k];
    int dj = j + diry[k];

    if(di < 0 || di >= n || dj < 0 || dj >= m)
        continue;

    int nxt_in = (di*m+dj)*2, nxt_out = (di*m+dj)*2 + 1;

    Edge* e1 = new Edge(), *e2 = new Edge();
    e1->nxt = nxt_in; e1->c = INF; e1->reverse = e2;
    e2->nxt = cur_out; e2->reverse = e1;

    stage[cur_out].push_back(e1);
    stage[nxt_in].push_back(e2);
}
```

```
int ans = 0;
while(1)
{
    memset(parent, -1, sizeof(parent));
    queue<int> q;
    parent[st] = st;
    q.push(st);

    while(!q.empty())
    {
        int cur = q.front(); q.pop();

        for(Edge* e : stage[cur])
            if(e->c - e->f > 0 && parent[e->nxt] == -1)
            {
                parent[e->nxt] = cur;
                path[e->nxt] = e;
                q.push(e->nxt);
            }
    }
```

```
if(parent[en] == -1)
    break;
```

```
int r = INF;
for(int p = en; p != st; p = parent[p])
    r = min(r, path[p]->c - path[p]->f);
```

```
for(int p = en; p != st; p = parent[p])
    path[p]->flow(r);
```

```
ans += r;
```

```

    }

    cout << (ans == INF ? -1 : ans) << '\n';
}

1.2 Biparate Matching
//백준 11375<열혈강호> 소스코드
vector<int> stage[1001];
vector<bool> status(1001);
int st[1001], en[1001];

bool dfs(int node)
{
    status[node] = 1;
    for(int x : stage[node])
    {
        if(en[x] == -1 || (!status[en[x]] && dfs(en[x])))
        {
            st[node] = x;
            en[x] = node;
            return true;
        }
    }

    return false;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, m;
    cin >> n >> m;

    for(int i = 1; i <= n; i++)
    {
        int k;
        cin >> k;
        while(k--)
        {
            int work;
            cin >> work;

            stage[i].push_back(work);
        }
    }

    int result = 0;
    fill(st, st+n+1, -1);
    fill(en, en+m+1, -1);

    for(int i = 1; i <= n; i++)
    {
        if(st[i] == -1)
        {
            fill(status.begin(), status.begin()+n+1, 0);

```

```

            if(dfs(i))
                result++;
        }
    }

    cout << result << '\n';
}

1.3 MCMF
//백준 11409<열혈강호 6> 소스코드
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

#define INF 0x7f7f7f7f

int c[805][805], f[805][805], cost[805][805];
vector<int> stage[805];
bool in[805];
int pre[805], dist[805];
queue<int> q;

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, m;
    cin >> n >> m;

    for(int i = 1; i <= n; i++)
    {
        stage[0].push_back(i);
        stage[i].push_back(0);
        c[0][i] = 1;
    }

    for(int i = 1; i <= m; i++)
    {
        stage[n+i].push_back(n+m+1);
        stage[n+m+1].push_back(n+i);
        c[n+i][n+m+1] = 1;
    }

    for(int i = 1; i <= n; i++)
    {
        int k;
        cin >> k;

        while(k--)
        {
            int a, b;
            cin >> a >> b;

            stage[i].push_back(n+a);
            stage[n+a].push_back(i);
            c[i][n+a] = 1;

```

```

        cost[i][n+a] = -b;
        cost[n+a][i] = b;
    }
}

int ans1 = 0, ans2 = 0;
while(1)
{
    fill(pre, pre+n+m+2, -1);
    fill(dist, dist+n+m+2, INF);
    dist[0] = 0;
    in[0] = 1;
    q.push(0);
    while(!q.empty())
    {
        int cur = q.front(); q.pop();
        in[cur] = 0;

        for(int nxt : stage[cur])
            if(c[cur][nxt]-f[cur][nxt] > 0 && dist[nxt] > dist[cur]+cost[cur][nxt])
            {
                dist[nxt] = dist[cur]+cost[cur][nxt];
                pre[nxt] = cur;
                if(!in[nxt])
                {
                    in[nxt] = 1;
                    q.push(nxt);
                }
            }
        }

        if(pre[n+m+1] == -1)
            break;

        int flow = INF;
        for(int i = n+m+1; i != 0; i = pre[i])
            flow = min(flow, c[pre[i]][i]-f[pre[i]][i]);

        ans1++;
        for(int i = n+m+1; i != 0; i = pre[i])
        {
            f[pre[i]][i] += flow;
            f[i][pre[i]] -= flow;
            ans2 += flow*cost[pre[i]][i];
        }
    }

    cout << ans1 << '\n' << -ans2 << '\n';
}

```

1.4 Bellman-Ford

//백준 11657<타임머신> 소스코드

#define INF 1e+18

#define ll long long

vector<pair<int, ll>> stage[501];

ll cost[501];

```

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, m;
    cin >> n >> m;

    while(m--)
    {
        int a, b; ll c;
        cin >> a >> b >> c;

        stage[a].push_back({b, c});
    }

    fill(cost, cost+n+1, INF);
    cost[1] = 0;
    bool flag = 0;

    for(int k = 0; k < n; k++)
        for(int i = 1; i <= n; i++)
            for(auto nxt : stage[i])
                if(cost[i] != INF && cost[nxt.first] > cost[i] + nxt.second)
                {
                    cost[nxt.first] = cost[i] + nxt.second;
                    if(k == n-1)
                        flag = 1;    //Is it cycle?
                }

    if(flag == 1)
        cout << "-1\n";
    else
        for(int i = 2; i <= n; i++)
            cout << (cost[i] != INF ? cost[i] : -1) << "\n";
}

```

1.5 SCC (Strongly Connected Component)

//백준 2150<Strongly Connected Component> 소스코드

int cnt = 1, scc_num;

vector<vector<int>> scc_result;

vector<int> stage[10005];

vector<bool> finished(10005, 0);

int dfsn[10005];

stack<int> s;

int dfs(int node)

```

{
    dfsn[node] = cnt++;
    s.push(node);

    int result = dfsn[node];
    for(int nxt : stage[node])
    {
        if(dfsn[nxt] == 0)
            result = min(result, dfs(nxt));
        else if(!finished[nxt])

```

```

        result = min(result, dfsn[nxt]);
    }

    if(result == dfsn[node])
    {
        vector<int> cur_scc;
        while(1)
        {
            int k = s.top();
            finished[k] = true;
            cur_scc.push_back(k);
            s.pop();

            if(k == node)
                break;
        }

        sort(cur_scc.begin(), cur_scc.end());
        scc_result.push_back(cur_scc);
        scc_num++;
    }

    return result;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int v, e;
    cin >> v >> e;

    while(e--)
    {
        int a, b;
        cin >> a >> b;

        stage[a].push_back(b);
    }

    for(int i = 1; i <= v; i++)
        if(dfsn[i] == 0)
            dfs(i);

    cout << scc_num << "\n";

    sort(scc_result.begin(), scc_result.end());
    for(auto& party : scc_result)
    {
        for(int x : party)
            cout << x << " ";
        cout << "-1\n";
    }
}

```

1.6 2-SAT

```

//백준 11281<2-SAT - 4> 소스코드
#include <iostream>
#include <vector>
#include <algorithm>
#include <stack>
using namespace std;

int cnt = 1, scc = 1;
vector<int> stage[20001];
int dfsn[20001];
vector<bool> finished(20001);
int scc_num[20001];
pair<int, int> v_scc[20001];
int ans[10001];

stack<int> s;

int dfs(int node)
{
    dfsn[node] = cnt++;
    s.push(node);

    int result = dfsn[node];
    for(int nxt : stage[node])
    {
        if(dfsn[nxt] == 0)
            result = min(result, dfs(nxt));
        else if(!finished[nxt])
            result = min(result, dfsn[nxt]);
    }

    if(result == dfsn[node])
    {
        while(1)
        {
            int k = s.top(); s.pop();
            scc_num[k] = scc;
            finished[k] = 1;

            if(k == node)
                break;
        }
        scc++;
    }

    return result;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, m;
    cin >> n >> m;
}

```

```

fill(ans, ans+n+1, -1);

while(m--)
{
    int x, y;
    cin >> x >> y;

    stage[-x+n].push_back(y+n);
    stage[-y+n].push_back(x+n);
}

for(int i = 0; i <= 2*n; i++)
    if(dfsn[i] == 0)
        dfs(i);

for(int i = 1; i <= n; i++)
    if(scc_num[-i+n] == scc_num[i+n])
    {
        cout << "0\n";
        return 0;
    }

cout << "1\n";
for(int i = 0; i <= 2*n; i++)
    v_scc[i] = {scc_num[i], i};

sort(v_scc, v_scc+2*n+1);

for(int i = 2*n; i >= 0; i--)
{
    int cur = v_scc[i].second;

    if(ans[abs(cur-n)] == -1)
    {
        if(cur-n < 0)
            ans[abs(cur-n)] = 1;
        else
            ans[cur-n] = 0;
    }
}

for(int i = 1; i <= n; i++)
    cout << ans[i] << " ";
cout << "\n";
}

```

2 Tree

2.1 HLD

```

//백준 13510<트리와 쿼리 1> 소스코드
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

#define X first

```

```

#define Y second

int num = 0;
vector<pair<int, int>> stage[100001];
vector<int> tree[100001];
int arr[100001], brr[100001];
int sz[100001], depth[100001], parent[100001];
int top[100001], in[100001];
pair<int, int> line[100001];

void make_tree(int cur, int pre = -1)
{
    for(auto nxt : stage[cur])
    {
        if(pre == nxt.X)
            continue;
        tree[cur].push_back(nxt.X);
        arr[nxt.X] = nxt.Y;
        make_tree(nxt.X, cur);
    }
}

void make_hld(int cur)
{
    sz[cur] = 1;
    for(int& nxt : tree[cur])
    {
        depth[nxt] = depth[cur] + 1;
        parent[nxt] = cur;
        make_hld(nxt);

        sz[cur] += sz[nxt];
        if(sz[nxt] > sz[tree[cur][0]])
            swap(nxt, tree[cur][0]);
    }
}

void dfs(int cur)
{
    in[cur] = ++num;
    for(int nxt : tree[cur])
    {
        top[nxt] = (nxt == tree[cur][0] ? top[cur] : nxt);
        dfs(nxt);
    }
}

void init(vector<int>& v, int node, int start, int end)
{
    if(start == end)
        v[node] = brr[start];
    else
    {
        init(v, node*2, start, (start+end)/2);
        init(v, node*2+1, (start+end)/2+1, end);
        v[node] = max(v[node*2], v[node*2+1]);
    }
}

```

```

    }
}

int query(vector<int>& v, int node, int start, int end, int left, int right)
{
    if(right < start || left > end)
        return 0;
    if(left <= start && end <= right)
        return v[node];

    int l = query(v, node*2, start, (start+end)/2, left, right);
    int r = query(v, node*2+1, (start+end)/2+1, end, left, right);
    return max(l, r);
}

void update(vector<int>& v, int node, int start, int end, int pos, int val)
{
    if(pos < start || pos > end)
        return;
    if(start == end)
    {
        v[node] = val;
        return;
    }

    update(v, node*2, start, (start+end)/2, pos, val);
    update(v, node*2+1, (start+end)/2+1, end, pos, val);
    v[node] = max(v[node*2], v[node*2+1]);
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;

    for(int i = 0; i < n-1; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;

        stage[u].push_back({v, w});
        stage[v].push_back({u, w});
        line[i] = {u, v};
    }

    make_tree(1);
    make_hld(1);
    top[1] = 1;
    dfs(1);

    for(int i = 1; i <= n; i++)
        brr[in[i]] = arr[i];

```

```

int h = (int)ceil(log2(n));
int tree_size = (1<<(h+1));
vector<int> segtree(tree_size);

init(segtree, 1, 1, n);

int q;
cin >> q;
while(q--)
{
    int x, a, b;
    cin >> x >> a >> b;

    if(x == 1)
    {
        a--;
        int u = line[a].X, v = line[a].Y;
        if(parent[u] == v)
            update(segtree, 1, 1, n, in[u], b);
        else
            update(segtree, 1, 1, n, in[v], b);
    }
    else
    {
        int ans = 0;
        for(; top[a] != top[b]; a = parent[top[a]])
        {
            if(depth[top[a]] < depth[top[b]])
                swap(a, b);
            ans = max(ans, query(segtree, 1, 1, n, in[top[a]], in[a]));
        }

        if(in[a] > in[b])
            swap(a, b);
        ans = max(ans, query(segtree, 1, 1, n, in[a]+1, in[b]));

        cout << ans << '\n';
    }
}
}

```

2.2 Centroid

```

//From Usaco Guide
#include <iostream>
#include <vector>

```

```
using namespace std;
```

```
const int maxn = 200010;
```

```
int n;
vector<int> adj[maxn];
int subtree_size[maxn];

```

```
int get_subtree_size(int node, int parent = -1) {
    int &res = subtree_size[node];
    res = 1;

```

```

    for (int i : adj[node]) {
        if (i == parent) { continue; }
        res += get_subtree_size(i, node);
    }
    return res;
}

int get_centroid(int node, int parent = -1) {
    for (int i : adj[node]) {
        if (i == parent) { continue; }

        if (subtree_size[i] * 2 > n) { return get_centroid(i, node); }
    }
    return node;
}

int main() {
    cin >> n;
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }

    get_subtree_size(0);
    cout << get_centroid(0) + 1 << endl;
}

```

3 Data Structure

3.1 Merge Sort Tree

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int arr[100001];
vector<int> MSTree[400001];

void init(int node, int start, int end)
{
    if (start == end)
        MSTree[node].push_back(arr[start]);
    else
    {
        init(node*2, start, (start+end)/2);
        init(node*2+1, (start+end)/2+1, end);

        vector<int> &l = MSTree[node*2], &r = MSTree[node*2+1];
        int lpos = 0, rpos = 0;
        while (lpos < l.size() || rpos < r.size())
        {
            if (rpos == r.size() || (lpos < l.size() && l[lpos] < r[rpos]))

```

```

            MSTree[node].push_back(l[lpos++]);
        else
            MSTree[node].push_back(r[rpos++]);
        }
    }
}

int query(int node, int start, int end, int left, int right, int val)
{
    if (right < start || left > end)
        return 0;
    if (left <= start && end <= right)
        return MSTree[node].end() - upper_bound(MSTree[node].begin(), MSTree[node].end(), val);

    int l = query(node*2, start, (start+end)/2, left, right, val);
    int r = query(node*2+1, (start+end)/2+1, end, left, right, val);
    return l+r;
}

3.2 Lazy Segment Tree (Sum ver.)
#include <cmath>
#define ll long long
void init(vector<ll>& a, vector<ll>& tree, int node, int start, int end)
{
    if (start == end)
        tree[node] = a[start];
    else
    {
        init(a, tree, node*2, start, (start+end)/2);
        init(a, tree, node*2+1, (start+end)/2+1, end);
        tree[node] = tree[node*2] + tree[node*2+1];
    }
}

void update_lazy(vector<ll>& tree, vector<ll>& lazy, int node, int start, int end)
{
    if (lazy[node] != 0)
    {
        tree[node] += (end-start+1) * lazy[node];
        if (start != end)
        {
            lazy[node*2] += lazy[node];
            lazy[node*2+1] += lazy[node];
        }
        lazy[node] = 0;
    }
}

void update_range(vector<ll>& tree, vector<ll>& lazy, int node, int start, int end, int left, int right, ll diff)
{
    update_lazy(tree, lazy, node, start, end);
    if (left > end || right < start)
        return;
    if (left <= start && end <= right)
    {
        tree[node] += (end-start+1) * diff;

```

```

        if(start != end)
        {
            lazy[node*2] += diff;
            lazy[node*2+1] += diff;
        }
        return;
    }

    update_range(tree, lazy, node*2, start, (start+end)/2, left, right, diff);
    update_range(tree, lazy, node*2+1, (start+end)/2+1, left, right, diff);
    tree[node] = tree[node*2] + tree[node*2+1];
}

ll query(vector<ll>& tree, vector<ll>& lazy, int node, int start, int end, int left, int
right)
{
    update_lazy(tree, lazy, node, start, end);
    if(left > end || right < start)
        return 0;
    if(left <= start && end <= right)
        return tree[node];

    ll lsum = query(tree, lazy, node*2, start, (start+end)/2, left, right);
    ll rsum = query(tree, lazy, node*2+1, (start+end)/2+1, end, left, right);
    return lsum + rsum;
}

```

3.3 Mo's Algorithm

//백준 13537<수열과 쿼리 1> 소스코드

```

#include <iostream>
#include <algorithm>
#include <cmath>
using namespace std;

int sn;
int arr[100001], res[100001];
int cnt[1000001];

struct Q {
    int l, r, i;

    bool operator< (Q& other) {
        if(l/sn != other.l/sn)
            return l/sn < other.l/sn;
        return r < other.r;
    }
};

Q query[100001];

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;

```

```

    sn = sqrt(n);

    for(int i = 1; i <= n; i++)
        cin >> arr[i];

    int q;
    cin >> q;
    for(int i = 0; i < q; i++)
    {
        cin >> query[i].l >> query[i].r;
        query[i].i = i;
    }

    sort(query, query+q);

    int ans = 0;
    int l = query[0].l, r = query[0].r;
    for(int i = 1; i <= r; i++)
    {
        if(cnt[arr[i]] == 0)
            ans++;
        cnt[arr[i]]++;
    }
    res[query[0].i] = ans;

    for(int i = 1; i < q; i++)
    {
        while(l < query[i].l)
        {
            cnt[arr[l]]--;
            if(cnt[arr[l]] == 0)
                ans--;
            l++;
        }
        while(l > query[i].l)
        {
            l--;
            if(cnt[arr[l]] == 0)
                ans++;
            cnt[arr[l]]++;
        }
        while(r < query[i].r)
        {
            r++;
            if(cnt[arr[r]] == 0)
                ans++;
            cnt[arr[r]]++;
        }
        while(r > query[i].r)
        {
            cnt[arr[r]]--;
            if(cnt[arr[r]] == 0)
                ans--;
            r--;
        }
    }
}

```

```

    res[query[i].i] = ans;
}

for(int i = 0; i < q; i++)
    cout << res[i] << '\n';
}

```

4 Strings

4.1 KMP

```

vector<int> get_failure(string& s)
{
    int sz = (int)s.size();
    vector<int> f(sz);

    int st = 1, match = 0;
    while(st+match < sz)
    {
        if(s[match] == s[st+match])
        {
            match++;
            f[st+match-1] = match;
        }
        else
        {
            if(match == 0)
                st++;
            else
            {
                st += (match - f[match-1]);
                match = f[match-1];
            }
        }
    }

    return f;
}

vector<int> kmp(string& s, string& t, vector<int>& f)
{
    int n = (int)s.size(), m = (int)t.size();
    vector<int> ret;

    int st = 0, match = 0;
    while(st+match <= n-m)
    {
        if(match < m && s[st+match] == t[match])
        {
            match++;
            f[st+match-1] = match;
        }
        else
        {
            if(match == 0)
                st++;
            else
            {

```

```

                st += (match - f[match-1]);
                match = f[match-1];
            }
        }
    }

    return ret;
}

4.2 Z
vector<int> GetZ(string& s)
{
    int sz = (int)s.size();
    vector<int> z(sz);

    int l = 0, r = 1;
    for(int i = 1; i < sz; i++)
    {
        if(i+z[i] < r)
            z[i] = min(z[i-1], r-i);
        while(i+z[i] < sz && s[z[i]] == s[i+z[i]])
            z[i]++;
        if(i+z[i] > r)
        {
            l = i;
            r = i+z[i];
        }
    }

    return z;
}

4.3 Manacher
string s, t = "!#";
cin >> s;

for(int i = 0; i < (int)s.size(); i++)
{
    t.push_back(tmp[i]);
    t.push_back('#');
}
t.push_back('@');

vector<int> GetP(string& s)
{
    int sz = (int)s.size();
    vector<int> p(sz);

    int l = 0, r = 1;
    for(int i = 1; i < sz-1; i++)
    {
        p[i] = max(0, min(p[i+(r-i)], r-i));
        while(s[i-p[i]] == s[i+p[i]])
            p[i]++;
        if(i+p[i] > r)
        {
            l = i-p[i];

```

```

        r = i+p[i];
    }
}

```

```
    return p;
```

```
}
```

4.4 Suffix Array & LCP

//백준 11479<서로 다른 부분 문자열의 개수 2> 소스 코드

```

#include <iostream>
#include <vector>
using namespace std;

```

```
#define ll long long
```

```
int cnt[1000005];
```

```
vector<int> get_sfx(string& s)
```

```

{
    int sz = (int)s.size();
    int limit = max(257, sz+1);

    vector<int> sfx(sz+1), group(sz+1), ngroup(sz+1), idx(sz+1);

    for(int i = 0; i < sz; i++)
    {
        group[i] = s[i];
        sfx[i] = i;
    }

    int t = 1;
    while(t < sz)
    {
        fill(cnt, cnt+limit, 0);
        for(int i = 0; i < sz; i++)
            cnt[group[min(i+t, sz)]]++;
        for(int i = 1; i < limit; i++)
            cnt[i] += cnt[i-1];
        for(int i = sz-1; i >= 0; i--)
            idx[--cnt[group[min(i+t, sz)]]] = i;

        fill(cnt, cnt+limit, 0);
        for(int i = 0; i < sz; i++)
            cnt[group[i]]++;
        for(int i = 1; i < limit; i++)
            cnt[i] += cnt[i-1];
        for(int i = sz-1; i >= 0; i--)
            sfx[--cnt[group[idx[i]]]] = idx[i];

        auto cmp = [&](int a, int b) {
            if(group[a] != group[b])
                return group[a] < group[b];
            return group[a+t] < group[b+t];
        };

        ngroup[sfx[0]] = 1;
        for(int i = 1; i < sz; i++)

```

```

        ngroup[sfx[i]] = ngroup[sfx[i-1]] + cmp(sfx[i-1], sfx[i]);
        group = ngroup;

```

```

        t *= 2;
    }

```

```
    return sfx;
```

```
}
```

```
vector<int> get_lcp(string& s, vector<int>& sfx)
```

```

{
    int sz = (int)s.size();
    vector<int> lcp(sz+1), rev(sz+1);
    for(int i = 0; i < sz; i++)
        rev[sfx[i]] = i;

    int k = 0;
    for(int i = 0; i < sz; i++)
    {
        if(rev[i] == sz-1)
        {
            k = 0;
            continue;
        }

        int j = sfx[rev[i]+1];
        while(i+k < sz && j+k < sz && s[i+k] == s[j+k])
            k++;
        lcp[rev[i]] = k;

        if(k > 0)
            k--;
    }

    return lcp;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    string s;
    cin >> s;

    int sz = (int)s.size();
    vector<int> sfx = get_sfx(s);
    vector<int> lcp = get_lcp(s, sfx);

    ll ans = 0;
    for(int i = 0; i < sz; i++)
        ans += (sz-i-lcp[i]);
    cout << ans << '\n';
}

```

5 Geometry

5.1 CCW

```
int ccw(Point a, Point b, Point c)
{
    ll val = (ll)(b.X-a.X)*(c.Y-b.Y) - (ll)(c.X-b.X)*(b.Y-a.Y);
    if(val > 0)
        return 1;
    if(val < 0)
        return -1;
    return 0;
}
```

5.2 Line Intersection

```
bool Intersect(Point a, Point b, Point c, Point d)
{
    int ab = ccw(a, b, c) * ccw(a, b, d);
    int cd = ccw(c, d, a) * ccw(c, d, b);
    if(ab == 0 && cd == 0)
    {
        if(a > b) swap(a, b);
        if(c > d) swap(c, d);
        return !(b < c || d < a);
    }

    return ab <= 0 && cd <= 0;
}
```

5.3 Convex Hull

```
//백준 6850<Cows> 소스코드
#include <iostream>
#include <vector>
#include <algorithm>
#include <stack>
using namespace std;
```

```
#define ll long long
#define X first
#define Y second
```

```
pair<int, int> cur;
pair<int, int> arr[10001];
vector<pair<int, int>> v;
stack<int> stk;
```

```
pair<int, int> vec(pair<int, int>& a, pair<int, int>& b)
{
    return {b.X-a.X, b.Y-a.Y};
}
```

```
ll cross(pair<int, int> a, pair<int, int> b)
{
    return (ll)(a.X*b.Y) - (ll)(b.X*a.Y);
}
```

```
int ccw(pair<int, int> a, pair<int, int> b, pair<int, int> c)
{
    ll val = (ll)(b.X-a.X)*(c.Y-b.Y) - (ll)(c.X-b.X)*(b.Y-a.Y);
```

```
    if(val > 0)
        return 1;
    if(val < 0)
        return -1;
    return 0;
}

ll dist(pair<int, int> a, pair<int, int> b)
{
    return (ll)(a.X-b.X)*(a.X-b.X) + (ll)(a.Y-b.Y)*(a.Y-b.Y);
}

bool cmp(pair<int, int>& a, pair<int, int>& b)
{
    ll val = ccw(cur, a, b);
    if(val != 0)
        return val > 0;
    return dist(cur, a) < dist(cur, b);
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;

    for(int i = 0; i < n; i++)
        cin >> arr[i].X >> arr[i].Y;

    sort(arr, arr+n);
    cur = arr[0];
    sort(arr+1, arr+n, cmp);

    stk.push(0);
    stk.push(1);
    for(int i = 2; i < n; i++)
    {
        while(stk.size() >= 2)
        {
            int b = stk.top(); stk.pop();
            int a = stk.top();

            if(ccw(arr[a], arr[b], arr[i]) > 0)
            {
                stk.push(b);
                break;
            }
        }

        stk.push(i);
    }

    while(!stk.empty())
    {
```

```

    v.push_back(arr[stk.top()]);
    stk.pop();
}

ll ans = 0;
for(int i = 1; i < (int)v.size()-1; i++)
    ans += cross(vec(v[0], v[i]), vec(v[0], v[i+1]));

cout << abs(ans) / 100 << '\n';
}

```

5.4 Rotating Callipus

//백준 9240<Robert Hood> 소스코드

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <stack>
#include <cmath>
using namespace std;

#define X first
#define Y second

pair<int, int> cur;
pair<int, int> arr[100001];
stack<int> stk;

int ccw(pair<int, int> a, pair<int, int> b, pair<int, int> c)
{
    int val = (b.X-a.X)*(c.Y-b.Y) - (c.X-b.X)*(b.Y-a.Y);
    if(val > 0) return 1;
    else if(val == 0) return 0;
    else return -1;
}

int dist(pair<int, int> a, pair<int, int> b)
{
    return (a.X-b.X)*(a.X-b.X) + (a.Y-b.Y)*(a.Y-b.Y);
}

int cross(pair<int, int> a, pair<int, int> b)
{
    return a.X*b.Y - a.Y*b.X;
}

pair<int, int> make_vector(pair<int, int> a, pair<int, int> b)
{
    return {b.X-a.X, b.Y-a.Y};
}

bool cmp(pair<int, int> a, pair<int, int> b)
{
    int val1 = ccw(cur, a, b);
    int val2 = ccw(cur, b, a);
    if(val1 != val2)
        return val1 > val2;
    return dist(cur, a) < dist(cur, b);
}

```

```

}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n;
    cin >> n;

    for(int i = 0; i < n; i++)
        cin >> arr[i].X >> arr[i].Y;

    if(n == 2)
    {
        cout << fixed;
        cout.precision(10);
        cout << sqrt(dist(arr[0], arr[1])) << '\n';
        return 0;
    }

    sort(arr, arr+n);
    cur = arr[0];
    sort(arr+1, arr+n, cmp);

    stk.push(0);
    stk.push(1);
    for(int i = 2; i < n; i++)
    {
        while(stk.size() > 1)
        {
            int b = stk.top(); stk.pop();
            int a = stk.top();

            if(ccw(arr[a], arr[b], arr[i]) > 0)
            {
                stk.push(b);
                break;
            }
        }

        stk.push(i);
    }

    vector<int> v;
    while(!stk.empty())
    {
        v.push_back(stk.top());
        stk.pop();
    }

    int sz = v.size(), a = 0, b = 1;
    int c = 1, d = 2;
    double ans = sqrt(dist(arr[v[0]], arr[v[1]]));
    while(1)

```

```

{
    if(cross(make_vector(arr[v[a]], arr[v[b]]), make_vector(arr[v[c]], arr[v[d]])) <= 0)
    {
        c = (c+1) % sz;
        d = (d+1) % sz;
        if(c == 1)
            break;
    }
    else
    {
        a = (a+1) % sz;
        b = (b+1) % sz;
        if(a == 0)
            break;
    }

    ans = max(ans, sqrt(dist(arr[v[a]], arr[v[c]])));
}

cout << fixed;
cout.precision(10);
cout << ans << '\n';
}

```

6 Extra

6.1 Sparse Table

```

//백준 17435<합성함수와 쿼리> 소스코드
//next[i][j] = 정점 i에서 2^j번 이동한 후의 정점
//initiating
for(int i = 1; i <= M; i++)
    cin >> next[i][0];

//i에서 2^(j+1)번 이동한 후의 정점은 i에서 2^j번*2번 이동하는 것
//next[i][j+1] = next[ next[i][j] ][j]
for(int j = 1; j < MAX_D; j++)
    for(int i = 1; i <= M; i++)
        next[i][j] = next[ next[i][j-1] ][j-1];

//processing query
while(q--)
{
    int n, x;
    cin >> n >> x;

    for(int j = MAX_D - 1; j >= 0; j--)
    {
        if(n >= (1<<j))
        {
            n -= (1<<j);
            x = next[x][j];
        }
    }

    cout << x << "\n";
}

```

6.2 Prefix Sum(Right Triangle Shape)

```

// 백준 5541<Nails> 소스코드
#include <iostream>
using namespace std;

int psum[5050][5050];

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    int n, m;
    cin >> n >> m;

    while(m--)
    {
        int a, b, x;
        cin >> a >> b >> x;

        psum[a][b]++;
        psum[a][b+1]--;
        psum[a+x+1][b]--;
        psum[a+x+1][b+x+2]++;
        psum[a+x+2][b+1]++;
        psum[a+x+2][b+x+2]--;

        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
                psum[i][j] += psum[i][j-1];

        for(int j = 1; j <= n; j++)
            for(int i = 1; i <= n; i++)
                psum[i][j] += psum[i-1][j];

        int ans = 0;
        for(int i = 1; i <= n; i++)
            for(int j = 1; j <= n; j++)
            {
                psum[i][j] += psum[i-1][j-1];
                ans += (psum[i][j] > 0);
            }

        cout << ans << '\n';
    }
}

6.3 Matrix Pow
//NOTICE! Max_size == 5
//NOTICE! mod == 1000

int n;
int stage[5][5];
int temp[5][5];
int result[5][5];

void multiple(int arr[5][5], int brr[5][5])

```

```

{
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            temp[i][j] = 0;

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            for(int p = 0; p < n; p++)
                temp[i][j] += (arr[i][p]*brr[p][j]) % 1000;
            temp[i][j] %= 1000;
        }

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            result[i][j] = temp[i][j];
}

void solve(int arr[5][5], long long k)
{
    if(k == 1)
    {
        multiple(result, arr);
        return;
    }

    solve(arr, k/2);
    multiple(result, result);
    if(k % 2 == 0)
        return;
    else
        multiple(result, arr);
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    long long b;
    cin >> n >> b;

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
        {
            cin >> stage[i][j];
            if(i == j)
                result[i][j] = 1;
        }

    solve(stage, b);

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
            cout << result[i][j] << " ";
    }
}

```

```

        cout << "\n";
    }
}

```

7 Tips/Tools

7.1 PBDS

#include <ext/pb_ds/assoc_container.hpp> // Common file
#include <ext/pb_ds/tree_policy.hpp> // Including tree_order_statistics_node_update
After closer inspection you may find that the last two files contained in the library

#include <ext/pb_ds/detail/standard_policies.hpp>
Namespace, which we will have to work in newer versions of C++ is called __gnu_pbds;, earlier it was called pb_ds;

Now let's look at the concrete structure.

The tree-based container has the following declaration:

```

template<
    typename Key, // Key type
    typename Mapped, // Mapped-policy
    typename Cmp_Fn = std::less<Key>, // Key comparison functor
    typename Tag = rb_tree_tag, // Specifies which underlying data structure to use
    template<
        typename Const_Node_Iterator,
        typename Node_Iterator,
        typename Cmp_Fn_,
        typename Allocator_>
        class Node_Update = null_node_update, // A policy for updating node invariants
        typename Allocator = std::allocator<char> > // An allocator type
    class tree;

```

Experienced participants may have already noticed that if initialize the template only the first two types, we obtain almost exact copy of the container map. Just say, that this container can be set, for this you just need to specify the second argument template type as null_type (in older versions it is null_mapped_type).

By the way Tag and Node_Update are missing in map. Let us examine them in more detail.

Tag is a class denoting a tree structure, which we will use. There are three base-classes provided in STL for this, it is rb_tree_tag (red-black tree), splay_tree_tag (splay tree) and ov_tree_tag (ordered-vector tree). Sadly, at competitions we can use only red-black trees for this because splay tree and OV-tree using linear-timed split operation that prevents us to use them.

Node_Update is a class denoting policy for updating node invariants. By default it is set to null_node_update, ie, additional information not stored in the vertices. In addition, C++ implemented an update policy tree_order_statistics_node_update, which, in fact, carries the necessary operations. Consider them. Most likely, the best way to set the tree is as follows:

```

typedef tree<
    int,
    null_type,
    less<int>,
    rb_tree_tag,

```

tree_order_statistics_node_update>

ordered_set;

If we want to get map but not the set, as the second argument type must be used mapped type. Apparently, the tree supports the same operations as the set (at least I haven't any problems with them before), but also there are two new features `it` is `find_by_order()` and `order_of_key()`. The first returns an iterator to the k-th largest element (counting from zero), the second `it` the number of items in a set that are strictly smaller than our item. Example of use:

```
ordered_set X;
X.insert(1);
X.insert(2);
X.insert(4);
X.insert(8);
X.insert(16);

cout<<*X.find_by_order(1)<<endl; // 2
cout<<*X.find_by_order(2)<<endl; // 4
cout<<*X.find_by_order(4)<<endl; // 16
cout<<(end(X)==X.find_by_order(6))<<endl; // true

cout<<X.order_of_key(-5)<<endl; // 0
cout<<X.order_of_key(1)<<endl; // 0
cout<<X.order_of_key(3)<<endl; // 2
cout<<X.order_of_key(4)<<endl; // 2
cout<<X.order_of_key(400)<<endl; // 5
```

7.2 Bitmasking

1. k번 비트가 1인지 0인지 확인 - `status & (1<<k) == (1<<k)`
2. k번 비트를 1로 만들기 - `status |= (1<<k)`
3. k번 비트를 0으로 만들기 - `status &= ~(1<<k)`
4. 모든 비트가 0인지 확인 - `status == 0`
5. 모든 비트가 1인지 확인 - `status == (1<<n)-1`

ex) 백준 2098<외판원 순회> 소스코드

```
#define INF 1e9+10

int n;
int stage[20][20];
int dp[20][(1<<16)+1];

int tsp(int cur, int status)
{
    int& ret = dp[cur][status];
    if(ret != -1)
        return ret;

    if(status == (1 << n)-1)
    {
        if(stage[cur][0] != 0)
            return stage[cur][0];
        return INF;
    }

    ret = INF;
    for(int i = 0; i < n; i++)
```

```
{
    if(status & (1 << i) || stage[cur][i] == 0)
        continue;
    ret = min(ret, tsp(i, status | (1 << i)) + stage[cur][i]);
}

return ret;
}

int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);

    cin >> n;

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            cin >> stage[i][j];

    memset(dp, -1, sizeof(dp));

    cout << tsp(0, 1) << "\n";
}
```

7.3 Memo

#sort -> a 뒤 b

#gcd -> (a < b)에서 (remain == 0 종료 / a = remain, b = a)

#전위순회(preorder) - 루트/왼/오

#중위순회(inorder) - 왼/루트/오

#후위순회(postorder) - 왼/오/루트

#(피보나치) 피사노주기 - 주기는 $M = 10^k$ ($k \geq 2$)일 때 $15 \cdot 10^{(k-1)}$

#교란수열 - $!n=(n1)(!n2)+!(n1)$ // $!0 = 1, !1 = 0$